Runbook — enabling the dormant security scaffolds

SECTION 06-operations-it TYPE runbook STATUS active OWNER Founder UPDATED 2026-06-16

The May-2026 red-team landed three hardenings as **dormant, flag-gated** scaffolds (PR #32). Each is **OFF by default** and changes nothing in production until you both (a) do the matching client / console work and (b) flip its flag. This runbook is the enable-when-ready procedure for each, in safe rollout order.

The fixes that are **already active** (forged-history injection demotion, the 48h subscription-expiry clamp, IP-keyed quota, captadel security headers, CI key scoping) need nothing here — they shipped on by default.

> **How these flags are set.** They are plain 2nd-gen function env vars, read the same way as `ADEL_DAILY_FREE`. Set them in `functions/.env` (loaded by firebase-functions at deploy) or on the Cloud Run service's env in the Google Cloud console, then **redeploy the affected function** — each flag is read at module load, so a redeploy (new revision) is required to take effect. Never commit real secrets to `functions/.env`; these three are non-secret booleans.

> **Rollback for all three is the same:** unset the flag (or set it back to the default) and redeploy. No data migration is destructive — `rcBindings` and the `functions/protected/` payloads are additive.

1. App Check on `/api/chat` — `ADEL_APPCHECK_MODE`

Closes: unauthenticated LLM endpoint (anyone can script `/api/chat` and burn the Gemini budget; CORS is not auth). **Affects function:** `chat`. **Values:** `off` (default) · `monitor` (verify-and-log, never block) · `enforce` (401 without a valid token).

What you need first

1. **Register App Check** in the Firebase console → App Check:
   - **Web** app → reCAPTCHA Enterprise (or reCAPTCHA v3) provider.
   - **iOS** app (Capacitor) → App Attest / DeviceCheck.
2. **Initialise App Check in the client** so requests carry the token. Web, in `assets/js/firebase-config.js` (or right after `initializeApp`):

```
import { initializeAppCheck, ReCaptchaEnterpriseProvider }
  from 'https://www.gstatic.com/firebasejs/<ver>/firebase-app-check.js';
initializeAppCheck(app, {
  provider: new ReCaptchaEnterpriseProvider('<RECAPTCHA_SITE_KEY>'),
  isTokenAutoRefreshEnabled: true,
});
```

`chat.js` already calls `/api/chat` via `fetch`; with App Check initialised, the Firebase SDK attaches `X-Firebase-AppCheck` automatically only when you use the callable/SDK transport. Since `/api/chat` is a plain `fetch`, fetch the token explicitly and add the header:

```
import { getToken } from '.../firebase-app-check.js';
const { token } = await getToken(appCheck, /* forceRefresh */ false);
headers['X-Firebase-AppCheck'] = token;
```

Do the equivalent in `captadel/public/assets/js/chat.js` if you also want App Check on the standalone service (it would need its own server-side check — not in this PR).

Rollout (monitor → enforce)

1. Deploy the client App Check init. **Leave `ADEL_APPCHECK_MODE` unset.**
2. Set `ADEL_APPCHECK_MODE=monitor` and redeploy `chat`. Watch logs: `appcheck monitor { result: 'valid' | 'absent' | 'invalid' }`.
3. When ~all real traffic logs `valid` (give it a few days + an app release cycle), set `ADEL_APPCHECK_MODE=enforce` and redeploy.

Verify

```
# enforce mode: a request with no token is rejected
curl -s -o /dev/null -w '%{http_code}\n' -X POST https://flygaca.com/api/chat \
  -H 'Content-Type: application/json' -d '{"message":"test"}'    # → 401
```

> **Do not jump straight to `enforce`.** Without console registration + a shipped client, enforce 401s every legitimate user. Monitor first, always.

---

2. RevenueCat identity binding — `ADEL_RC_REQUIRE_BINDING`

Closes: the IAP webhook grants Pro to whatever `app_user_id` the client chose, so a buyer could attach Pro to an arbitrary account. **Affects function:** `revenuecatWebhook` (+ the `linkRevenueCatIdentity` callable, already deployed). **Values:** unset/`off` (default — grants on `app_user_id` as today) · truthy (`1`/`true`) — grants **only** for an `app_user_id` with a server-side binding.

What you need first

1. **Ship the client link call.** In the iOS sign-in path (where `assets/js/native-bridge.js` calls `Purchases.logIn(uid)`), first call the callable so the binding exists before any purchase:

   ```
   import { getFunctions, httpsCallable } from '.../firebase-functions.js';
   const fns = getFunctions(app, 'me-central1');
   await httpsCallable(fns, 'linkRevenueCatIdentity')();   // writes rcBindings/{uid}
   await Purchases.logIn({ appUserID: uid });
   ```

   This is idempotent — safe to call on every sign-in.

2. **Backfill existing subscribers** so live Apple subs don't lose access when you flip the flag. One-time Admin-SDK script (run locally with a service account), creating a binding for every uid that currently holds an App-Store entitlement:

   ```
   // backfill-rcbindings.js  (node, firebase-admin)
   const admin = require('firebase-admin'); admin.initializeApp();
   const db = admin.firestore();
   const snap = await db.collection('users')
     .where('entitlement.store', '==', 'APP_STORE').get();   // RC-sourced grants
   const batch = db.batch();
   snap.forEach((d) ⇒ batch.set(db.collection('rcBindings').doc(d.id),
     { uid: d.id, linkedAt: admin.firestore.FieldValue.serverTimestamp(), backfilled:
   true },
     { merge: true }));
   await batch.commit();
   console.log('backfilled', snap.size, 'bindings');
   ```

   (If `entitlement.store` isn't indexed/queryable, iterate all users and filter in code.)

Rollout

1. Deploy the client link call (it's harmless while the flag is off — it just writes `rcBindings`).
2. Run the backfill once.
3. Set `ADEL_RC_REQUIRE_BINDING=1` and redeploy `revenuecatWebhook`.

Verify

- A new purchase from a signed-in account grants Pro (binding exists).
- In logs, an event for an unbound id shows `revenuecat grant skipped — no identity binding` and returns `200 skipped:unbound` (acknowledged, not granted — RevenueCat won't retry-storm).

> `rcBindings` is a **server-only** collection — `firestore.rules` denies all client access and the Admin SDK bypasses rules, so a client can neither read nor forge a binding. No rules change is needed.

---

3. Server-side paywall — `ADEL_PROTECTED_CONTENT`

Closes: every "Pro" payload (question bank, Ground School, Prep-Pack bodies) ships as a **public static file** — `curl` the asset URL and the paywall is moot. `gate.js` only blurs the DOM after the data already loaded. **Affects function:** `protectedContent` (`/api/content`). **Values:** unset/ `off` (default — endpoint returns `501`) · truthy — token + entitlement gate live.

> **Product decision first.** Your code currently has `WEB_GATING_LIVE=false` — content is *intentionally* open pre-launch. **Do not enable this until you actually want to lock the paywall.** Enabling it without the client cutover below would just leave the new endpoint serving `501` while the static files stay public — no worse, but no better. The value comes from doing the whole cutover together.

Cutover (do all of it, then flip the flag — see also `functions/protected/README.md`)

1. **Move payloads server-side** into `functions/protected/` and add them to the `CONTENT` allowlist in `functions/content.js` :
   - `assets/data/quiz.json` → `functions/protected/quiz.json` (id `quiz`)
   - `assets/data/groundschool.json` → `functions/protected/groundschool.json` (id `groundschool`)
   - each Prep-Pack body → `functions/protected/pack-<id>.json` (ids `pack-aip` , ...)
2. **Stop publishing them** — add `assets/data/quiz.json` , `assets/data/groundschool.json` , and the gated `packs/**` pages to `hosting.ignore` in `firebase.json` . (The ignore list is a publish filter, not access control — the real protection is steps 1 + 4. Decide your free-preview slice and keep that public.)
3. **Update the client** to fetch through the gate WITH the ID token, and render only on `200` (handle `401` / `403` with the paywall prompt). In `assets/js/study.js` , `assets/js/groundschool.js` , and the pack pages:

```
import { getAuth } from '.../firebase-auth.js';
const idToken = await getAuth().currentUser?.getIdToken();
const res = await fetch('/api/content?id=quiz',
  { headers: idToken ? { Authorization: 'Bearer ' + idToken } : {} });
if (res.status === 401 || res.status === 403) { showPaywall(); return; }
const data = await res.json();
```

4. **Enable** — set `ADEL_PROTECTED_CONTENT=1` , deploy functions **and** hosting together (so the static files disappear and the gate goes live in one shot).

Verify

```
# anonymous: no token → 401 (was: full JSON)
curl -s -o /dev/null -w '%{http_code}\n' https://flygaca.com/api/content?id=quiz   # 401
# the old static URL is gone
curl -s -o /dev/null -w '%{http_code}\n' https://flygaca.com/assets/data/quiz.json # 404
# a signed-in Pro user (paste a fresh ID token) gets 200
curl -s -o /dev/null -w '%{http_code}\n' -H "Authorization: Bearer $ID_TOKEN" \
  https://flygaca.com/api/content?id=quiz                                        # 200
```

Quick reference

Flag	Default	Function	Enable after
`ADEL_APPCHECK_MODE`	`off`	`chat`	App Check registered + client init; `monitor` → `enforce`
`ADEL_RC_REQUIRE_BINDING`	`off`	`revenuecatWebhook`	client `linkRevenueCatIdentity` shipped + backfill run
`ADEL_PROTECTED_CONTENT`	`off`	`protectedContent`	payloads moved server-side + client sends ID token + `firebase.json` ignore updated

All three roll back by unsetting the flag and redeploying.